Documentación y análisis de rendimiento
Algoritmos iterativos vs recursivos en Java

Fecha: 03/03/2026

Este documento describe la implementación de cuatro algoritmos (factorial, fibonacci, búsqueda lineal y ordenamiento burbuja) en dos variantes: iterativa y recursiva. Para cada caso se midió el tiempo de ejecución (5 repeticiones) para distintos tamaños de entrada (n), y se compararon los resultados con la complejidad temporal teórica (Big-O).

La evidencia numérica y las gráficas se generaron a partir de las mediciones registradas en el archivo Excel de resultados del proyecto.

1. Objetivo y alcance

Objetivo general: comparar, de forma cuantitativa, el desempeño entre implementaciones iterativas y recursivas para cuatro algoritmos clásicos, y justificar su complejidad temporal mediante evidencia empírica (mediciones + gráficas) y análisis teórico.

Alcance: se consideraron cuatro algoritmos, cada uno implementado en Java en dos variantes (iterativa y recursiva). Se realizaron 5 ejecuciones por tamaño de entrada (n) y se utilizó el promedio como métrica principal de comparación.

Limitaciones: (1) Las mediciones se realizaron con temporización de propósito general (System.nanoTime), por lo que existe variabilidad por el JIT de la JVM, cachés y recolección de basura. (2) El factorial se calcula en tipo *long*, por lo que se limita a $n \leq 20$ para evitar desbordamiento; por ello no se reportan datos para n=25 y n=30.

2. Algoritmos implementados

A1 - Factorial: Calcula n! multiplicando los enteros de 1 a n. La versión recursiva usa la definición $n! = n \cdot (n-1)!$ con caso base $0!=1$.

A2 - Fibonacci: Calcula F(n). La versión iterativa acumula dos valores previos en un ciclo. La versión recursiva usa $F(n)=F(n-1)+F(n-2)$ (sin memoización).

A3 - Búsqueda lineal: Busca un valor en un arreglo recorriéndolo secuencialmente. La versión recursiva evalúa el índice i y avanza i+1 hasta encontrar o terminar.

A4 - Burbuja: Ordena un arreglo comparando e intercambiando adyacentes. La versión recursiva reduce el rango a ordenar tras cada pasada.

3. Metodología de medición

Para cada algoritmo y cada tamaño de entrada n se ejecutaron 5 repeticiones, registrando el tiempo transcurrido en milisegundos (ms) y calculando el promedio. La medición se realizó con System.nanoTime() antes y después de la llamada al algoritmo; el tiempo se obtuvo como (fin–inicio) y se convirtió a ms. Con el fin de mantener comparabilidad, la misma secuencia de n se utilizó para las versiones iterativa y recursiva.

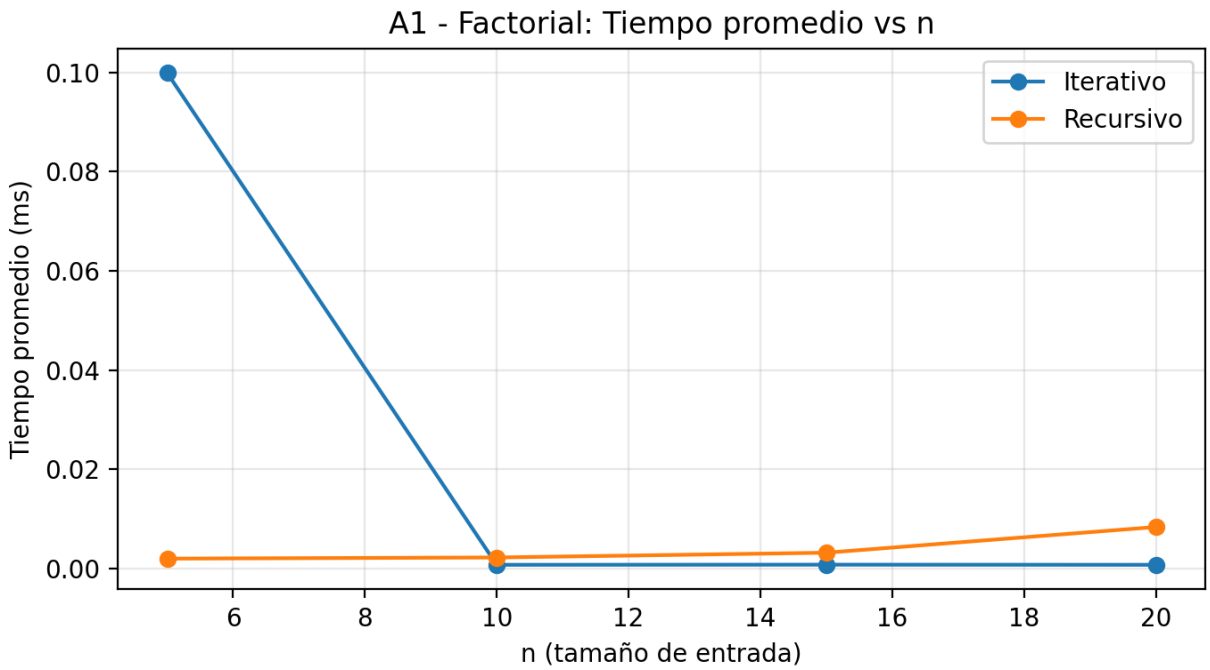
Consideración importante: la primera ejecución de un método puede ser más lenta debido a la compilación JIT y calentamiento de la JVM. En este reporte se conserva el esquema solicitado de 5 ejecuciones y promedio, y se discute el impacto de esta variación en el análisis.

4. Resultados experimentales

Las siguientes subsecciones presentan (a) una tabla resumen con el tiempo promedio por n y (b) una gráfica comparativa Iterativo vs Recursivo. En Fibonacci se utiliza escala logarítmica en el eje Y para visualizar el crecimiento acelerado de la versión recursiva.

4.1 A1 - Factorial

n	Iterativo (ms)	Recursivo (ms)
5	0.100	0.001981
10	0.000737	0.002215
15	0.000763	0.003177
20	0.000736	0.008355

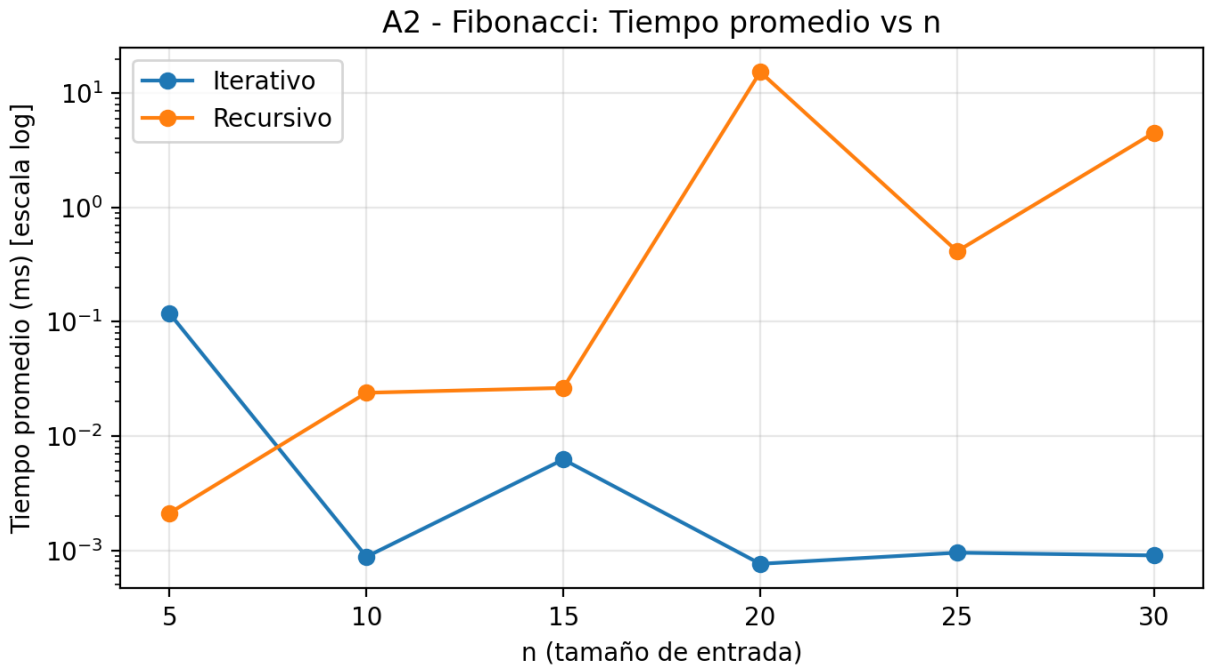


Observación: ambos enfoques muestran tiempos muy bajos para n≤20. La recursión agrega un costo constante por llamadas y manejo de pila, pero no cambia el orden de crecimiento esperado (lineal).

4.2 A2 - Fibonacci

n	Iterativo (ms)	Recursivo (ms)
5	0.118	0.002123
10	0.000880	0.024
15	0.006243	0.026
20	0.000763	15.339

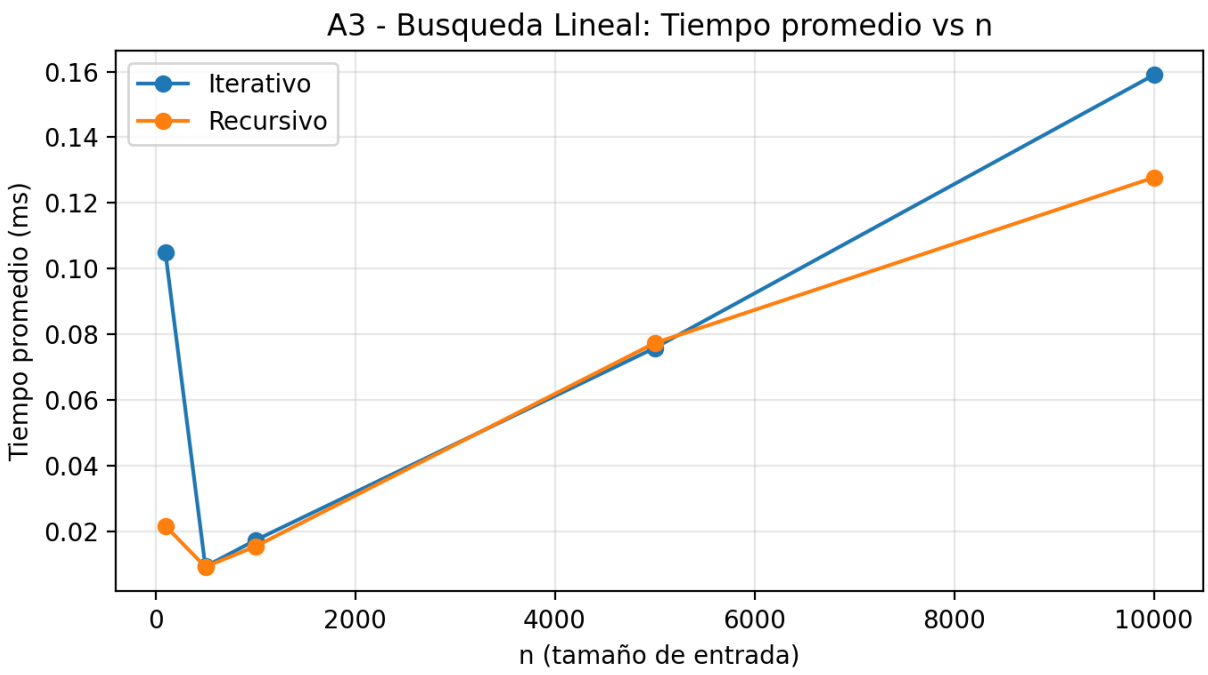
25	0.000955	0.414
30	0.000904	4.511



Observación: la versión recursiva crece de forma desproporcionada con n, consistente con un comportamiento exponencial. La versión iterativa crece suavemente y mantiene tiempos bajos, consistente con crecimiento lineal.

4.3 A3 - Busqueda Lineal

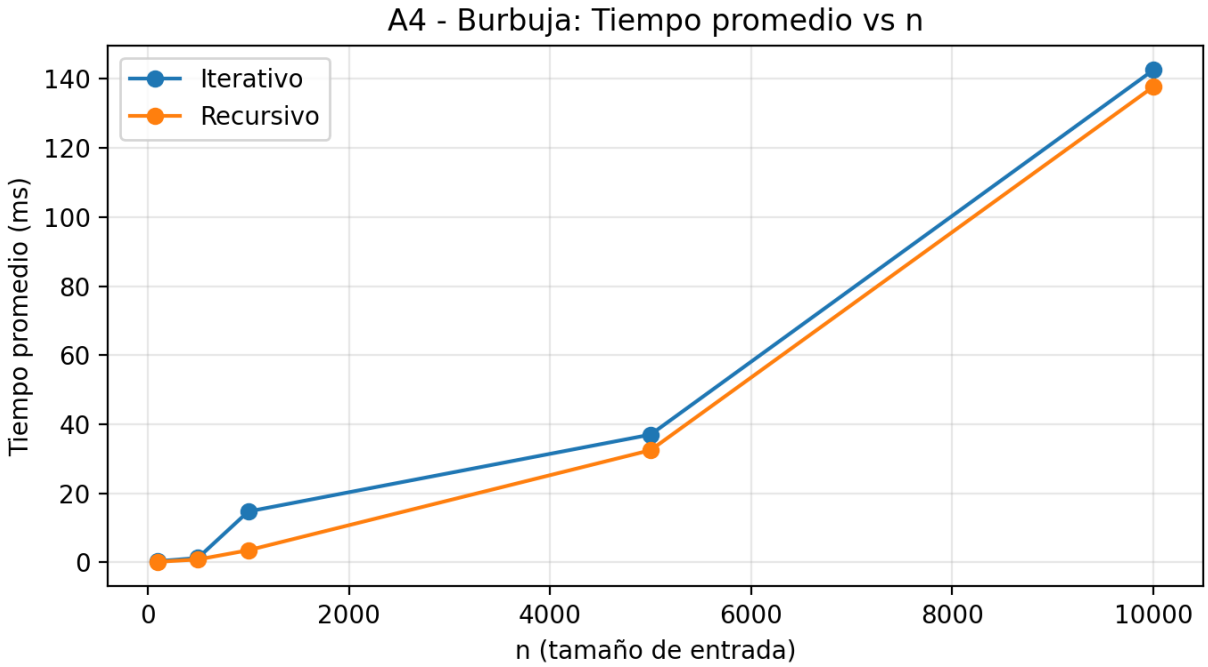
n	Iterativo (ms)	Recursivo (ms)
100	0.105	0.022
500	0.009303	0.009217
1000	0.017	0.015
5000	0.076	0.077
10000	0.159	0.128



Observación: tanto iterativo como recursivo crecen aproximadamente de forma lineal conforme aumenta n. La versión recursiva tiende a ser ligeramente más lenta por sobrecarga de llamadas.

4.4 A4 - Burbuja

n	Iterativo (ms)	Recursivo (ms)
100	0.323	0.140
500	1.263	0.825
1000	14.718	3.492
5000	36.977	32.472
10000	142.587	137.747



Observación: el crecimiento es pronunciado conforme aumenta n, compatible con un algoritmo cuadrático. La versión recursiva no mejora el orden; mantiene el mismo patrón O(n^2) con una constante adicional.

5. Análisis de complejidad temporal (Big-O) y justificación

Factorial: Tanto la variante iterativa como la recursiva realizan aproximadamente n multiplicaciones. Por ello, el número de operaciones crece proporcionalmente a n, lo que corresponde a O(n). En las mediciones, los tiempos permanecen en el orden de micro/milisegundos; la recursión agrega una constante por llamadas, pero no altera el orden de crecimiento.

Fibonacci: La variante recursiva clásica (sin memoización) recalcula subproblemas repetidamente; el número de llamadas crece aproximadamente como O(2^n) (más precisamente relacionado con ϕ^n). Esto explica el incremento abrupto observado. En contraste, la versión iterativa calcula cada término una sola vez, con O(n) operaciones, consistente con su curva estable.

Búsqueda lineal: En el peor caso se inspeccionan n elementos del arreglo. Esto implica O(n) tanto iterativo como recursivo; la diferencia radica en una constante adicional por llamadas y verificación de caso base.

Burbuja: Ejecuta en el orden de n(n–1)/2 comparaciones/intercambios, equivalente a O(n^2). La variante recursiva conserva la misma cantidad de comparaciones, por lo que mantiene O(n^2). El crecimiento al aumentar n respalda esta expectativa.

5.1 Variabilidad y validez de las mediciones

Se observa variabilidad entre repeticiones, especialmente cuando una ejecución es significativamente mayor. Este fenómeno es consistente con el calentamiento de la JVM (compilación JIT) y con efectos del sistema operativo (planificación, cachés, frecuencia de CPU). Para un análisis micro-benchmark más estricto se recomienda: (a) warm-up explícito, (b) fijar afinidad de CPU, (c) desactivar turbo, y (d) usar JMH. Aun con estas limitaciones, los datos son suficientes para distinguir tendencias lineales, cuadráticas y exponenciales, que es el objetivo del ejercicio.

6. Conclusiones

• En factorial, búsqueda lineal y burbuja, la recursión no modifica el Big-O respecto a la versión iterativa; la diferencia principal es una constante adicional por llamadas y uso de pila.

• Fibonacci muestra que una recursión ingenua puede ser computacionalmente inviable: sin memoización el costo crece exponencialmente, mientras que la versión iterativa mantiene crecimiento lineal.

• Para entradas grandes, la versión iterativa suele ser preferible por eficiencia y por evitar riesgo de desbordamiento de pila; la recursión se justifica cuando aporta claridad y no compromete complejidad.

• Las gráficas respaldan la teoría: factorial y búsqueda lineal son compatibles con O(n), burbuja con O(n^2), y fibonacci recursivo con crecimiento compatible con O(2^n).

Anexo A. Archivos generados

El proyecto exporta un CSV con las 5 ejecuciones y el promedio por combinación (algoritmo, versión, n). Este CSV alimenta la hoja de datos crudos del Excel y permite reproducir las gráficas.

CSV: resultados/tiempos.csv